

OpenSpec 工作原理讨论

2026-08-19 16:22

Table of Contents

1. OpenSpec 是什么？核心业务是什么？
 2. 它有没有类似"知识库/RAG"那样把代码库塞进 prompt 的功能？
 3. spec 是怎么初始化的？
 4. 核心循环：维护 spec $\rightarrow$ propose change $\rightarrow$ apply $\rightarrow$ 更新 spec
 5. 把 OpenSpec 拆成"init 准备"和"运行时注入规范"两部分来看
- 第 1 部分：准备阶段 (init)
- 第 2 部分：运行阶段 (AI 调用 skill 时)
6. 两个核心文件：schema 和 config
- 两者的角色：正交的两个维度
- 两者如何被合并使用
- 第 1 步：定位并加载 schema (决定工作流结构)
- 第 2 步：分别取"工作流规范"和"项目规范"
- 第 3 步：组装成对象 $\rightarrow$ 输出 JSON
7. 工程师能自定义工作流的哪些部分？
 8. OpenSpec 可优化和提升的方向 (讨论)
- 8.1 规格与代码的一致性 (drift)
 - 8.2 Markdown 约定的"静默失败"
 - 8.3 规格更新时机偏晚 (只在 archive)
 - 8.4 多人协作与并发冲突
- 附：一张图理解整体数据流

1. OpenSpec 是什么？核心业务是什么？

OpenSpec 是一个服务于人机协作写代码的"Spec驱动开发" workflow 引擎：它的主要任务是与工程师合作，为项目维护一个 spec。它为 AI 工具制定 proposal、spec、design、task 等文档的书写规范与产出顺序；用 spec 作为项目的事实来源来约束 AI 的思考和实现，并让每次变更最终沉淀回 spec，成为闭环。

它管理三样东西的生命周期：

概念	含义	存放位置
Specs	项目当前的"真相"，描述系统应有哪些能力/需求	<code>openspec/specs/</code>
Changes (变更)	一次待办的 Spec 修改提案（为什么改 + 改什么）	<code>openspec/changes/</code>
Archive (归档)	已完成并合并进 Specs 的变更历史	<code>openspec/changes/archive/</code>

围绕这个闭环，还有两个配套业务：

- 为 30+ 种 AI 工具（Claude、Cursor、Copilot 等）生成集成指令（skill 和 slash 命令）。
- 校验 Markdown Spec 文件的格式是否合法。

2. 它有没有类似"知识库/RAG"那样把代码库塞进 prompt 的功能？

没有。这是理解 OpenSpec 时最关键的一点。

OpenSpec 不做代码库索引、向量化（embedding）、分块（chunking）或检索（retrieval），也没有向量库和 RAG。

那"代码库怎么进入 prompt"？——不是 OpenSpec 干的，是 AI 工具（Claude Cursor）自己干的。OpenSpec 的角色是 "prompt 脚手架 工作流编排器"：

- `init` 时把一批指令模板写成 AI 工具能识别的文件（例如 `.claude/skills/`、`.cursor/commands/` 里的 `/opsx:*` 命令）。
- 当你触发 `/opsx:propose` 时，这些指令被注入 prompt，告诉 AI：“去读 `openspec/specs/` 下的相关文件，去搜索代码里的 TODO，去看 `git log`……”。
- 真正读代码、把内容拉进上下文的动作，是 AI Agent 用它自己的文件读取/搜索工具完成的。

所以，"知识"的载体是人和 AI Agent 手写的 Markdown Spec 文件——它是"人工维护的知识库"，而非"自动抽取的代码索引"。

3. spec 是怎么初始化的？

OpenSpec 初始化时不会遍历代码库。`openspec init` 只做三件事：

- 建立空目录：`openspec/specs/`、`openspec/changes/`、`openspec/changes/archive/`（specs 初始是空的）。
- 生成项目配置 `openspec/config.yaml`（内容只是引用一个内置 schema，见第一行 `schema: spec-driven`）。
- 按检测到的 AI 工具生成对应的 skill 和 `/opsx:*` 命令文件。

它不扫描源代码、不建索引、不生成 spec 内容。

那 spec 从哪来？两条途径：

- 正常的 **change** → **archive** 循环：spec 随着一个个变更被归档而逐渐“长”出来。
- ****onboard** / **explore** 等由 AI 驱动的工作流：让 AI 读代码后手写 spec —— 但这依然是大模型在做，不是确定性的扫描器。

4. 核心循环：维护 spec → propose change → apply → 更新 spec

该循环的工作流由 schema 定义（见 `schemas/spec-driven/schema.yaml`），描述为：`proposal → specs → design → tasks`。完整生命周期：

阶段	干什么
propose	生成 change 容器 + <code>proposal.md</code> (Why / What Changes) + 增量 spec (delta specs)
specs	写增量段落： <code>ADDED / MODIFIED / REMOVED / RENAMED Requirements</code>
design / tasks	技术设计文档 + 任务清单（用 <code>- []</code> 复选框驱动进度）
apply	AI 逐条实现任务、勾选 <code>- [x]</code>
archive	校验 → 把增量合并回正式 spec → 变更目录移入 <code>archive/</code>

一个容易搞错的细节：更新 spec 不是在 **apply** 阶段，而是在 **archive** 阶段。**apply** 只改代码、勾任务；真正把 change 里的增量 (delta) 合并进 `openspec/specs/`，是 **archive** 这一步做的。

合并遵循固定顺序 **RENAMED** → **REMOVED** → **MODIFIED** → **ADDED**，并在合并前做冲突预检（同一需求不能同时出现在多个操作段；重命名时 **MODIFIED** 必须引用新名字，等等）。

5. 把 OpenSpec 拆成“init 准备”和“运行时注入规范”两部分来看

第 1 部分：准备阶段 (init)

- 建立 `openspec/` 目录结构。
- 按检测到的 AI 工具生成对应的 skill 和 `/opsx:` 命令。
- 生成 `openspec/config.yaml`（含项目级 context / rules，并引用一个内置 schema）。

注意：`schema.yaml` 不是 init 生成的，它随 npm 包内置发布（见 `schemas/spec-driven/schema.yaml`），全局共用；`config.yaml` 只是用 `schema: spec-driven` 引用它。只有主动运行 `openspec schema fork`，项目里才会出现一份可自定义的本地 schema。

第 2 部分：运行阶段 (AI 调用 skill 时)

目标是“让 AI 按 schema + config 的规范去产出”。这里的关键是——规范不是直接塞进 prompt，而是分两层、延迟到运行时拉取：

- 1. 静态注入的只是 skill 文本（流程说明书）：你敲 `/opsx:propose` 时被注入 prompt 的，是那段 skill 指令文本，它只讲"操作流程"，不含 schema 里的具体规范。
- 2. 规范由 AI 运行时调 CLI 拉取：skill 要求 AI 执行 `openspec instructions <artifact> --change <name> --json`（以及 `openspec status`）。这些命令在后台读取 `schema.yaml` + `config.yaml` 拼成 JSON 返回，AI 再据此产出 `proposal spec design / tasks`。

这样设计的好处：skill 保持精简（只讲流程），真正的规范延迟到运行时动态拼装——改了 schema 或 config 后无需重新生成 skill，规范就能立即生效。

至于这份 JSON 具体怎么拼、每个字段来自哪，见下一节。

6. 两个核心文件：schema 和 config

OpenSpec 运行时的"规范"来自两个 YAML 文件：`schema.yaml` 和 `config.yaml`。理解它们各自的角色，是看懂整个工作流的关键。

这里是这两个文件的中文翻译版 `schema.yaml` 和 `config.yaml` 可供参考

两者的角色：正交的两个维度

这两个文件互不重叠，各管一个维度——一个定义"流程长什么样"，一个定义"这个项目有什么特殊要求"。

维度	<code>schema.yaml</code> （工作流模型）	<code>config.yaml</code> （项目模型）
回答的问题	一次变更要产出哪些文档、按什么顺序、每份怎么写	这个项目在写这些文档时有哪些背景和额外约束
作用域	跨项目复用（包内置 用户级 项目级 三级）	绑定单个项目
稳定性	相对稳定，是一套可复用的"方法论模板"	随项目技术栈/团队规范变化
典型内容	artifacts 列表、依赖关系（ <code>requires</code> ）、每个 artifact 的 <code>instruction</code> 和 <code>template</code> 、 <code>apply</code> 阶段配置	<code>context</code> （项目背景）、按 artifact 分类的 <code>rules</code> 、多仓库 <code>references</code>
打个比方	通用的"公文写作规范手册"	贴在手册边上的"本部门补充规定"

- `**schema.yaml` = 工作流的骨架（HOW / 通用）：定义整条流水线的形状。换掉 schema 就等于换一套方法论（比如从"提案驱动"换成"测试先行"），与具体项目无关。
- `**config.yaml` = 项目的约束（WHAT here / 专属）：不改变流程形状，只往每一步注入"本项目特有的背景和规则"，例如"本工具跨平台，路径必须用 `path.join()`"。

两者如何被合并使用

在运行时，`openspec instructions` 命令负责把这两个正交维度按当前 artifact 交叉合并成一份 JSON，供 AI 使用。它不是把两个文件"拼成一段文本"，而是从两个来源分别取字段，组装成结构化对象，再序列化成为 JSON。分三步：

第 1 步：定位并加载 schema（决定工作流结构）

根据 change 的元数据或 config 里的 `schema:` 名字解析出 schema，构建成 artifact 依赖图。schema 查找有三级优先顺序（这正是自定义 schema 的入口，高优先级覆盖低优先级）：

- 1. 项目级：`<项目>/openspec/schemas/<name>/schema.yaml`
- 2. 用户级：`${XDG_DATA_HOME}/openspec/schemas/<name>/schema.yaml`
- 3. 包内置：`<包>/schemas/<name>/schema.yaml`

第 2 步：分别取" workflow规范"和"项目规范"

- 从 `schema` 取：当前 artifact 的模板（`templates/*.md`）、依赖关系、写作指导（`instruction`）、描述、产出路径。
- 从 `openspec/config.yaml` 取：`context`（项目背景）和当前 artifact 对应的 `rules`。

一个关键设计：`**context` 和 `rules` 是独立字段，不会被塞进 `template` 文本里——目的是让 AI 明白"这些是约束，别抄进产出文件"。

第 3 步：组装成对象 → 输出 JSON

返回的 JSON 里，每个字段的来源如下：

JSON 字段	来源	含义
<code>instruction</code>	schema.yaml 的 artifact 指导	这类 artifact 该怎么写（workflow层规范）
<code>template</code>	schema 的 <code>templates/<file>.md</code>	产出文件的结构骨架
<code>description</code> <code>outputPath</code> <code>dependencies</code> / <code>unlocks</code>	schema.yaml	描述、产出路径、依赖关系
<code>context</code>	config.yaml 的 <code>context</code>	项目背景约束（技术栈、跨平台要求等）
<code>rules</code>	config.yaml 的 <code>rules[artifactId]</code>	该 artifact 的项目专属规则
<code>changeName</code> <code>changeDir</code> <code>planningHome</code> / <code>resolvedOutputPath</code>	运行时路径解析	具体写到哪

一句话概括：schema 提供"通用 workflow规范 + 模板"，config 提供"你项目的专属约束"，两者按 artifact 维度组装成一份 JSON；AI 拿到后照着 `template` 写，把 `instruction` `context` `rules` 当约束。

7. 工程师能自定义 workflow的哪些部分？

想改什么	怎么改
项目的约束（最常用）	编辑 <code>openspec/config.yaml</code> ： <code>schema</code> （用哪套流程）、 <code>context</code> （项目背景）、 <code>rules</code> （按 artifact 分类的规则）、 <code>references/store</code> （多仓库，Beta）
workflow本身（产出物 顺序 模板）	<code>openspec schema fork spec-driven my-flow</code> 生成项目本地 schema 再改：增删 artifacts、调整 <code>instruction/template/requires</code> 、配置 <code>apply.requires</code> 与 <code>tracks</code> ；改完把 config 的 <code>schema</code> ：指向它

想改什么	怎么改
跨项目通用流程	把 schema 放到 <code>\${XDG_DATA_HOME}/openspec/schemas/<name>/</code> ，对所有项目生效
哪些 AI 工具、生成哪些命令	重跑 <code>openspec init</code> <code>update</code> 增删工具； <code>openspec config profile</code> 选 workflow 集合； <code>delivery</code> 决定生成 <code>skills.commands</code> / 两者

优先级：项目本地 schema > 用户级 > 包内置。参考内置示例 `schemas/spec-driven/schema.yaml`。spec 内容和 change 提案则是日常直接编写的正文。

8. OpenSpec 可优化和提升的方向（讨论）

以下是基于其架构的一些思考，部分可能已被官方部分实现或列入计划，供讨论。

8.1 规格与代码的一致性（drift）

- **问题**：spec 是纯人工维护的“真相”，但没有确定性机制检测“代码是否已改、spec 是否更新”（或反向）。时间一长，spec 容易与真实代码脱节，失去“事实来源”的价值。
- **可提升**：建立 spec ↔ 代码的双向可追溯与漂移检测，具体可以做：
 - **建立映射关系**：允许在 requirement/scenario 里声明它对应的代码符号或测试（如 `@covers: src/core/archive.ts#ArchiveCommand`、`@test: test/archive.spec.ts`），并在 spec 侧维护一份可校验的追溯索引。
 - **新增 `openspec drift` 命令**：扫描映射两端做一致性检查，输出可分级的报告——**未被任何测试覆盖的 scenario**、**spec 引用的代码符号已不存在（重命名/删除）**、**最近改动的文件没有关联到任何 requirement**。
 - **接入 git 做增量判断**：对比上次归档以来的 `git diff`，找出“改了代码但没有对应活跃/已归档 change”的模块，以及“改了 spec 但代码未跟进”的反向漂移。
 - **CI 门禁 + 分级严重度**：以 `--strict` 在 CI 中将高危漂移（如引用了已删除符号）判为失败、低危（如覆盖率下降）仅告警，并输出机器可读的 `--json` 供流水线消费。
 - **量化与可视化**：把结果沉淀为“规格覆盖率/新鲜度”指标（多少 requirement 有测试、多少 scenario 长期未验证），与 8.7 的健康度看板打通。

8.2 Markdown 约定的“静默失败”

- **问题**：合并高度依赖 Markdown 结构的精确匹配——scenario 必须恰好 `####`（4 个井号）、MODIFIED 必须复制整块并保证 header 完全一致，否则会静默丢内容（schema 内置说明里已明确警告这些坑）。这对新手很不友好。
- **可提升**：加强 `validate` 的 lint 能力，把这些“静默失败”变成显式警告并给出行号与修复建议；对 MODIFIED 提供“从现有 spec 自动抽取原块”的辅助命令，减少手工复制出错。

8.3 规格更新时机偏晚（只在 archive）

- **问题**：正式 spec 只有在 `archive` 时才被更新。一个长期不归档的 change，其最新决策无法反映到 specs，其他人看到的仍是旧规格；且合并是“全量、一次性”的。
- **可提升**：让 `apply` 阶段支持增量、可回滚地把已定稿的 delta 预同步到 specs（`specs-apply` 已具备底层能力），或提供“草稿态 spec 视图”降低归档前的信息滞后。

8.4 多人协作与并发冲突

- 问题：多个 change 同时修改同一个 spec 时，归档顺序不同会相互覆盖，缺少语义层的冲突检测与合并策略。
- 可提升：归档前检测"目标 requirement 是否已被其他已归档 change 改动"，冲突时给出三方对比而非直接失败或覆盖。

附：一张图理解整体数据流



整个过程中，AI 每要产生一个 artifact，就调用一次 `openspec instructions` 拉取"schema 规范 + config 约束"合并成的 JSON，据此生成符合规范的内容。